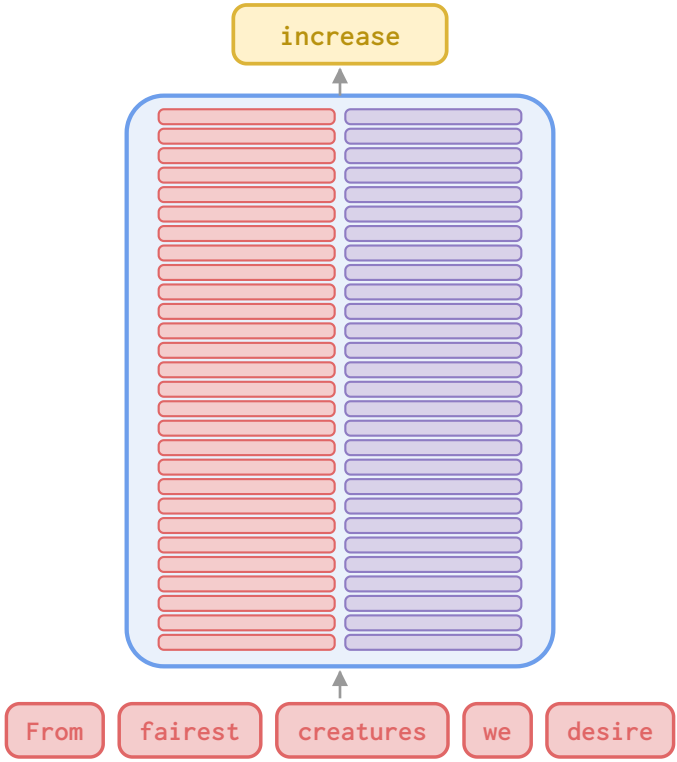


UNIT 01 · CHAPTER 13

Building irishQwen

CSE 10124 · Building ChatGPT

William Theisen · University of Notre Dame

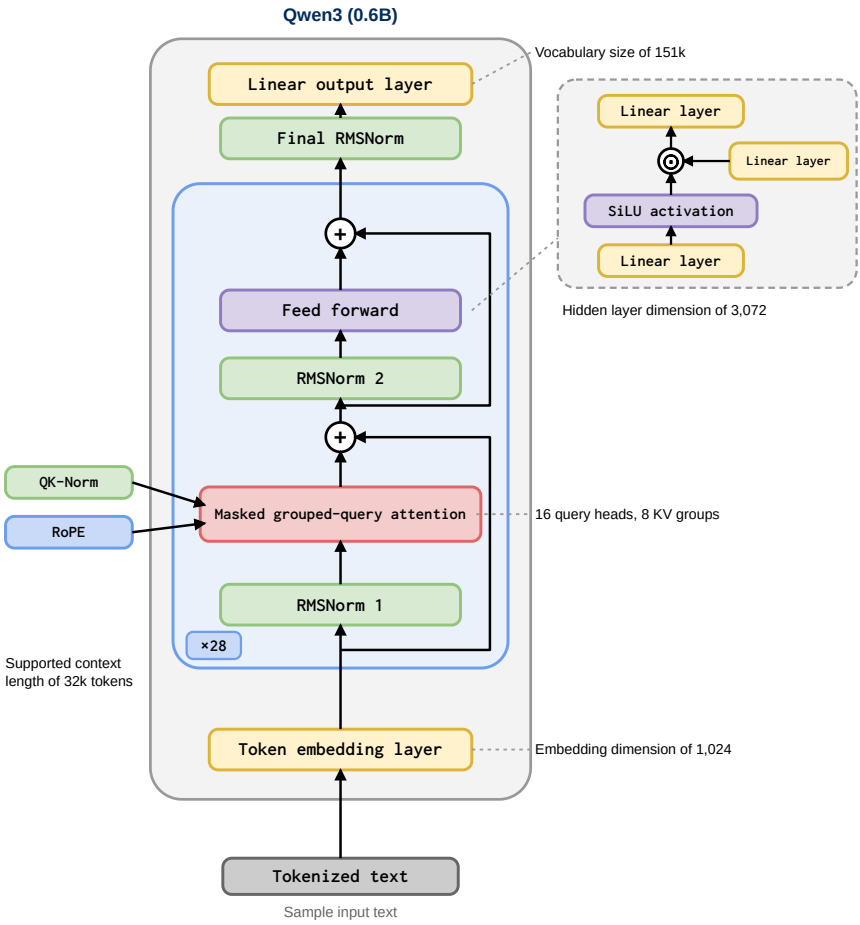
Building irishQwen

Last chapter ended with every piece of irishQwen on the table, an embedding table, Qwen3 blocks, a final RMSNorm and an output layer, all with random weights, so the probabilities coming out of the top meant nothing.

All that's left is to put the pieces together into one model, count what is in it, train it on Shakespeare and ask it to write. The model is **irishQwen**, the class's own copy of Qwen3, the family of open-weight models Alibaba released in `2025`. irishQwen has exactly the design of **Qwen3-0.6B**, the smallest of them.

Our running example is the first line of Shakespeare's first sonnet, "From fairest creatures we desire increase," which we will follow from text to token ids to a guess at what comes next. The rest of Shakespeare is what we train on.

The Qwen3-0.6B Architecture



Reading from the bottom, text becomes token ids, each id becomes a vector of `1,024` numbers, and those vectors go up through `28` identical blocks before a final RMSNorm and the output layer turn each one into a guess at the next token. Each block has grouped-query attention, with `16` query heads sharing `8` key/value heads, and a SwiGLU feed-forward network `3,072` wide. **Nothing in it is new**, it is just all of it at once.

irishQwen's Files

irishQwen lives in a GitHub repository, with one Python file per piece: `rms_norm.py`, `rotary_embedding.py`, `linear_layer.py`, `silu.py`, `swiglu.py`, `grouped_query_attention.py`, `embedding.py`, `qwen_transformer.py` (one block) and `irishQwen.py`, which stacks them all into a model.

Each file holds one layer as a class with three methods. `forward` is the calculation we have been drawing all along. `backward` takes the gradient of the loss with respect to the layer's output and works out the gradient for the layer's own weights and the gradient for its input (handed down to the layer below). `update` moves the weights a small step against their gradients, the same gradient descent step from Optimization.

Writing every backward pass by hand is the point of the class code, it's the only way to be sure we know what each layer does. Several of those methods are left empty as exercises, so **the repository won't train until they're filled in**. Each student forks the repository (makes a copy of their own on GitHub) and fills them in there.

The rest of this chapter builds the same model in a few pages of code with PyTorch's own layers, which work out every backward pass for us.

Token Ids

Everything starts with text. irishQwen's Shakespeare tokenizer is the byte-level BPE tokenizer from the BPE chapter, trained on the complete works with a vocabulary of only **512** tokens: the **256** bytes, three special tokens (<|pad|>, <|sos|> and <|eos|>) and **253** merges. The opening line of the first sonnet, **42** characters, becomes **22** tokens (spaces drawn as ·):



With only **253** merges the tokens stay short, mostly a pair of letters or a letter with the space before it. The longest are a handful of common words like `·shall` and `·love`, and "fairest" takes four tokens. Over the whole corpus (with the Project Gutenberg notices taken out), **4,875,049** characters become **2,526,266** tokens, about **1.93** characters a token.

Qwen3-0.6B's own tokenizer has about **151,700** tokens (its embedding table has room for **151,936**), enough to cover most languages and most programming languages, where most common English words are a single token. A small vocabulary keeps our model small, at the price of longer sequences.

A Browser-Sized irishQwen

The code in this book runs in your browser, on a version of PyTorch that does all of its arithmetic on the processor, with no graphics card. Unfortunately that makes it slow, so **our browser model is tiny**. It skips the tokenizer and reads raw bytes (every byte is its own token, so $V = 256$), its vectors have `32` numbers instead of `1,024`, and it has `2` blocks instead of `28`. Every layer is the real one, only the sizes are small.

Here are the three sizes of irishQwen in this chapter, Qwen3-0.6B's, the class's Shakespeare model and ours:

	Qwen3-0.6B	Shakespeare	Browser
Vocabulary V	`151,936`	`512`	`256`
Vector size C	`1,024`	`256`	`32`
Blocks L	`28`	`4`	`2`
Query / key-value heads	`16` / `8`	`8` / `4`	`4` / `2`
Head size	`128`	`32`	`8`
SwiGLU width	`3,072`	`768`	`64`
Window (tokens)	`32,768`	`128`	`32`

The middle column is the training script's defaults, the size the class trains in Google Colab on a T4 graphics card. The last row is how many tokens each one reads at once.

The Embedding Layer

We start from `20,000` bytes of the sonnets and an embedding table. `nn.Embedding` is the $V \times C$ table from the Embeddings chapter, as a layer PyTorch can train:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

f = open("shakespeare.txt", "rb")
f.seek(10462)                    # where THE SONNETS start
data = f.read(20000)             # a 20 kB slice
ids = torch.tensor(list(data))
print(data[64:76])
print(ids[64:76].tolist())

torch.manual_seed(0)
embed = nn.Embedding(256, 32)  # V = 256 bytes, C = 32
X = embed(ids[64:96].view(1, -1))  # one window
print(X.shape)
```

OUTPUT

```
b'From fairest'
[70, 114, 111, 109, 32, 102, 97, 105, 114, 101, 115, 116]
torch.Size([1, 32, 32])
```

The ids are just bytes (`F` is `70`, a space is `32`). One window of `32` bytes, "From fairest creatures we desire", comes out of the table as a $1 \times 32 \times 32$ tensor: a batch of one window, `32` tokens, `32` numbers each.

RMSNorm and RoPE as Layers

A layer PyTorch can train is a class that extends `nn.Module`, with its weights wrapped in `nn.Parameter` so PyTorch knows they are there to be learned. RMSNorm has one such weight, `gamma`, a scale for each dimension that starts at `1`. RoPE has no weights at all (it is a fixed rotation), so it can stay a plain function:

```
class RMSNorm(nn.Module):
    def __init__(self, d):
        super().__init__()
        self.gamma = nn.Parameter(torch.ones(d))

    def forward(self, x):
        ms = (x * x).mean(dim=-1, keepdim=True)
        return self.gamma * x / torch.sqrt(ms + 1e-6)

def rope(x, theta=10000.0):    # x: (B, H, T, Dh)
    T, Dh = x.shape[-2], x.shape[-1]
    i = torch.arange(0, Dh, 2).float()
    pos = torch.arange(T).float()
    ang = torch.outer(pos, theta ** (-i / Dh))
    ang = torch.cat([ang, ang], dim=-1)
    x1, x2 = x.chunk(2, dim=-1)
    turned = torch.cat([-x2, x1], dim=-1)
    return x * torch.cos(ang) + turned * torch.sin(ang)
```

Both are the calculations from last chapter. RMSNorm divides each vector by its root mean square and then scales it by `gamma`. RoPE turns each pair of numbers in a query or key by an angle that grows with the token's position, quickly for the first pairs and more and more slowly for the later ones.

Grouped-Query Attention

Attention is the longest layer. It follows irishQwen's `grouped_query_attention.py`, with four query heads sharing two key and value heads:

```python
class GQA(nn.Module):
    def __init__(self, C, H, H_kv, Dh):
        super().__init__()
        self.H, self.H_kv, self.Dh = H, H_kv, Dh
        self.Wq = nn.Linear(C, H * Dh, bias=False)
        self.Wk = nn.Linear(C, H_kv * Dh, bias=False)
        self.Wv = nn.Linear(C, H_kv * Dh, bias=False)
        self.Wo = nn.Linear(H * Dh, C, bias=False)
        self.q_norm = RMSNorm(Dh)
        self.k_norm = RMSNorm(Dh)
    def heads(self, x, n):  # -> (B, n, T, Dh)
        B, T, _ = x.shape
        return x.view(B, T, n, self.Dh).transpose(1, 2)
    def forward(self, x):
        B, T, C = x.shape
        q = self.heads(self.Wq(x), self.H)
        k = self.heads(self.Wk(x), self.H_kv)
        v = self.heads(self.Wv(x), self.H_kv)
        q = rope(self.q_norm(q))
        k = rope(self.k_norm(k))
        G = self.H // self.H_kv  # query heads per K/V
        k = k.repeat_interleave(G, dim=1)
        v = v.repeat_interleave(G, dim=1)
        s = q @ k.transpose(-1, -2) / self.Dh ** 0.5
        late = torch.triu(torch.ones(T, T), diagonal=1)
        s = s.masked_fill(late.bool(), float("-inf"))
        ctx = F.softmax(s, dim=-1) @ v
        ctx = ctx.transpose(1, 2).reshape(B, T, -1)
        return self.Wo(ctx)
```

Grouped-Query Attention, Step by Step

Reading `forward` from the top, each token vector is projected into queries, keys and values and split into heads of `Dh` numbers. With four query heads and only two key/value heads, `Wk` and `Wv` are half as wide as `Wq`, and each key/value head is copied (`repeat_interleave`) so it serves `G = 2` query heads.

Queries and keys then get QK-Norm and RoPE, and the scores are taken. The scores for later tokens (everything above the diagonal, `late`) are set to minus infinity, so softmax gives them a weight of exactly zero and **no token can look ahead at the text it is about to predict**. Finally the heads are glued back together and mixed by `Wo`.

```
attn = GQA(C=32, H=4, H_kv=2, Dh=8)
print(attn(X).shape)
```

OUTPUT

```
torch.Size([1, 32, 32])
```

The same shape comes out that went in, one window of **32** vectors of **32** numbers, ready to be added back onto the residual stream.

SwiGLU

The other half of a block is the feed-forward network, SwiGLU. Two linear layers look at the same token vector. One of them (`up`) says what could get through, the other (`gate`), after a SiLU, says how much of it does. Multiplying the two element by element and projecting back down with a third layer gives a vector the size we started with:

```python
class SwiGLU(nn.Module):
    def __init__(self, C, F_dim):
        super().__init__()
        self.gate = nn.Linear(C, F_dim, bias=False)
        self.up = nn.Linear(C, F_dim, bias=False)
        self.down = nn.Linear(F_dim, C, bias=False)

    def forward(self, x):
        g = F.silu(self.gate(x))  # how much gets through
        u = self.up(x)            # what gets through
        return self.down(g * u)

z = torch.tensor([-4.0, -1.28, 0.0, 1.0, 3.0])
print(F.silu(z))
print(SwiGLU(32, 64)(X).shape)
```

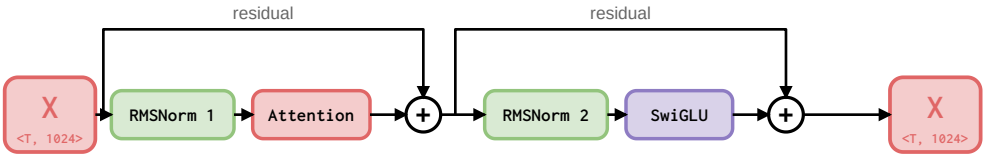
OUTPUT

```
tensor([-0.0719, -0.2785,  0.0000,  0.7311,  2.8577])
torch.Size([1, 32, 32])
```

SiLU (`z` times the sigmoid of `z`) lets large positive values through almost unchanged (`3` becomes `2.86`) and squashes negative ones toward zero, though not all the way: its lowest point is `-0.28`, at $z = -1.28$. The middle layer is twice as wide as the vector here, three times in Qwen3-0.6B.

The Qwen3 Block

A Qwen3 block runs RMSNorm, attention, an add, then RMSNorm, SwiGLU and another add. Each half normalizes its input first (pre-norm) and adds its result back onto the vectors that came in (the residual connection):



In code that is four layers and two lines:

```python
class Block(nn.Module):
    def __init__(self, C, H, H_kv, Dh, F_dim):
        super().__init__()
        self.ln1, self.ln2 = RMSNorm(C), RMSNorm(C)
        self.attn = GQA(C, H, H_kv, Dh)
        self.ffn = SwiGLU(C, F_dim)

    def forward(self, x):
        x = x + self.attn(self.ln1(x))
        x = x + self.ffn(self.ln2(x))
        return x
```

The two `x = x + …` lines are the residual stream. **Each half of the block only adds to the vectors coming up**, it never replaces them, which is what lets **28** of these stack without the signal fading away.

The Whole Model

irishQwen is then an embedding table, a stack of blocks, a final RMSNorm and an output layer (`lm_head`) that turns each vector back into a score for every token in the vocabulary:

```
class IrishQwen(nn.Module):
    def __init__(self, V, C, L, H, H_kv, Dh, F_dim):
        super().__init__()
        self.embed = nn.Embedding(V, C)
        blocks = [Block(C, H, H_kv, Dh, F_dim)
                  for _ in range(L)]
        self.blocks = nn.ModuleList(blocks)
        self.final_ln = RMSNorm(C)
        self.lm_head = nn.Linear(C, V, bias=False)

    def forward(self, ids):              # (B, T)
        x = self.embed(ids)              # (B, T, C)
        for block in self.blocks:
            x = block(x)                 # (B, T, C)
        return self.lm_head(self.final_ln(x))  # (B, T, V)

torch.manual_seed(0)
model = IrishQwen(V=256, C=32, L=2, H=4, H_kv=2,
                  Dh=8, F_dim=64)
logits = model(ids[64:96].view(1, -1))
print(logits.shape)
```

OUTPUT

```
torch.Size([1, 32, 256])
```

One window of **32** bytes goes in and 32×256 scores come out. **Every position gets its own guess at the token after it**, so one window holds **32** predictions, not one. That will matter as soon as we train.

Next-Token Probabilities

The scores are logits, and a softmax turns the last position's logits into a probability for each of the **256** bytes. Our window stopped at "we desire", and Shakespeare's next byte is a space (then "increase"). Here are the model's three favorites:

```
probs = F.softmax(logits[0, -1], dim=-1)
p, nxt = probs.topk(3)
print(round(probs.sum().item(), 4))
print([bytes([i]) for i in nxt.tolist()])
print([round(x, 4) for x in p.tolist()])
```

OUTPUT

```
1.0
[b'K', b'x', b'\xd0']
[0.0188, 0.0171, 0.0147]
```

The probabilities add up to **1**, as they should, and the favorites are K, x and a byte that is only half of a two-byte character. None of them is a space. Its top pick gets under **2%**, against **0.4%** for every byte if it spread its bets evenly, so with random weights **the model has no idea**.

Training has to fix that, by setting every one of the model's numbers. It is worth knowing how many there are first.

Counting Parameters

Every number training is allowed to change is a **parameter**. We can count them from the shapes alone. Each linear layer has its input size times its output size (Qwen3 uses no biases), each RMSNorm has one number per dimension, and the embedding table has one row of `C` numbers per token:

```
def count(V, C, L, H, H_kv, Dh, F_dim, tied):
    attn = 2 * C * H * Dh + 2 * C * H_kv * Dh + 2 * Dh
    block = attn + 3 * C * F_dim + 2 * C
    head = 0 if tied else C * V
    return V * C + L * block + C + head

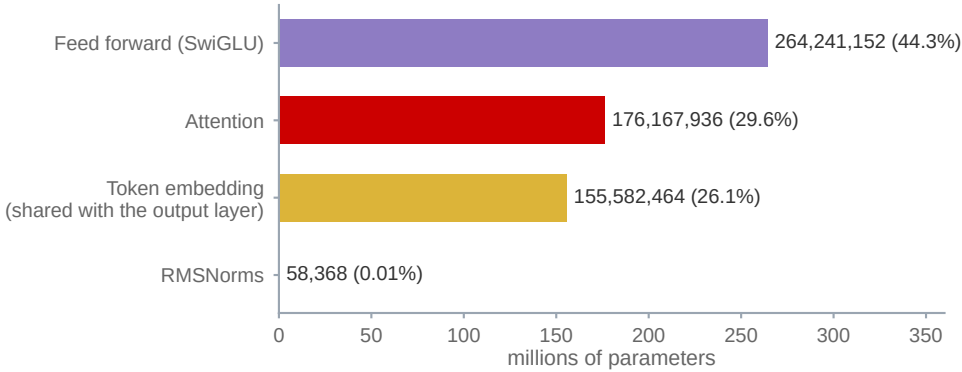
ours = sum(p.numel() for p in model.parameters())
print(ours, count(256, 32, 2, 4, 2, 8, 64, False))
print(count(151936, 1024, 28, 16, 8, 128, 3072, True))
```

OUTPUT

```
35008 35008
596049920
```

Our browser model has **`35,008`** parameters, and the formula agrees with PyTorch's own count. Put Qwen3-0.6B's sizes into the same formula (**`151,936`** tokens, vectors of **`1,024`**, **`28`** blocks, **`16`** query and **`8`** key/value heads of **`128`**, SwiGLU **`3,072`** wide) and we get **`596,049,920`**, the "0.6B" in its name. The last argument, `tied`, is about the output layer, which we'll get to.

Where the Parameters Live



Pictured above is where Qwen3-0.6B's parameters live. The feed-forward networks hold the most, **44.3%**, since every block has three SwiGLU matrices of $1{,}024 \times 3{,}072$. Attention holds **29.6%**, less than you might expect for the famous part, partly because grouped-query attention halves the key and value matrices. The **57** RMSNorms together hold **58,368** numbers, about **0.01%**.

That leaves the embedding table, a single table of $151{,}936 \times 1{,}024$ holding **26.1%** of the model. A big vocabulary is expensive. Every token Qwen3 knows, in every language it reads, costs another **1,024** numbers.

Tied Embeddings

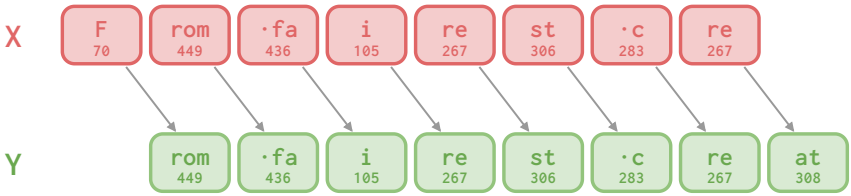
The embedding table turns a token id into a vector, and the output layer turns a vector back into a score for every token. The two are the same size, `V × C`, one the transpose of the other. Qwen3-0.6B uses **one table for both jobs**, which is called **tied embeddings**: a token's vector going in is also the direction that scores it coming out.

Untied, Qwen3-0.6B would need a second table of `155,582,464` numbers, `751,632,384` in all, a quarter more. For a small model with a big vocabulary that is a lot to spend on the vocabulary, so the small Qwen3 models tie their tables and the larger ones (8B and up) don't bother.

irishQwen keeps two separate tables, but its Shakespeare vocabulary has only `512` tokens, so they hardly matter. With the training script's sizes (vectors of `256`, `4` blocks, `8` query and `4` key/value heads, SwiGLU `768` wide) it has `3,410,432` parameters, and the two tables are `262,144` of them, `7.7%`. Almost everything is in the blocks.

Training Windows

Training needs examples with answers, and a language model makes its own from the text. The training script turns the whole corpus into one long list of **2,526,266** token ids and cuts it into windows of **129** tokens. The first **128** are the input X and the last **128** are the labels Y, the same tokens shifted along by one (here the first **9** tokens of the sonnet):



Every token in X has the token after it as its label, so **one window is 128 next-token questions at once**, and attention's mask makes sure no position can peek at its own answer. The last **5%** of the corpus is held back for later, which leaves **18,749** windows to train on and **986** to check with.

The Training Loop

Training is the loop from the Optimization chapter: a batch of windows forward, the cross-entropy against the labels, the gradients, an Adam step. In the browser we use `32`-byte windows, batches of `8` and only `100` steps (it still takes a little while):

```
T, B = 32, 8
starts = torch.arange(0, len(ids) - T - 1, T)
opt = torch.optim.Adam(model.parameters(), lr=3e-3)
torch.manual_seed(1)
for step in range(101):
    pick = starts[torch.randint(0, len(starts), (B,))]
    rows = [ids[s:s + T + 1] for s in pick.tolist()]
    W = torch.stack(rows)
    X, Y = W[:, :-1], W[:, 1:]     # Y is X shifted by one
    logits = model(X)
    loss = F.cross_entropy(logits.reshape(-1, 256),
                           Y.reshape(-1))
    opt.zero_grad()
    loss.backward()
    opt.step()
    if step % 25 == 0:
        print(step, round(loss.item(), 3))
```

OUTPUT

```
0 5.668
25 3.175
50 2.714
75 2.699
100 2.365
```

`loss.backward()` does what irishQwen's `backward` methods do by hand. The loss starts at `5.668`, a little worse than `ln 256 = 5.545`, the loss of a model that spreads its bets evenly over every byte. After `100` steps it is down to `2.365`.

Validation Loss

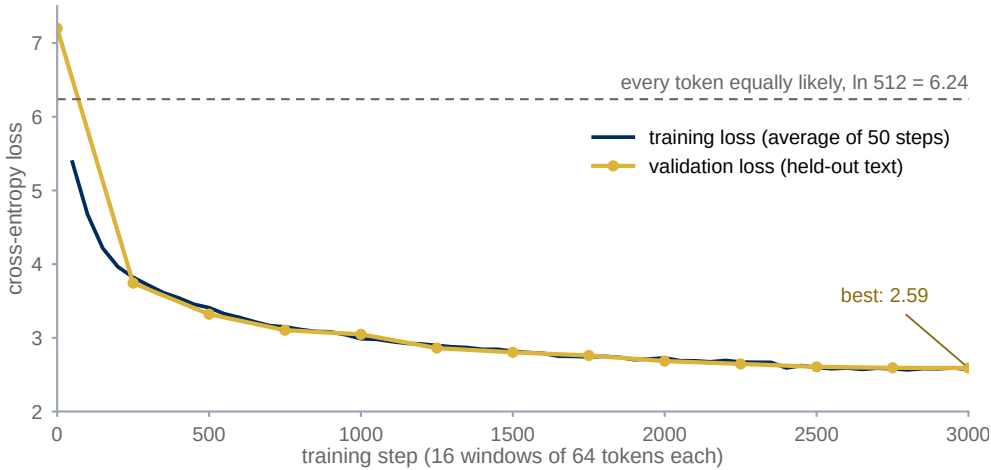
A falling training loss only says the model is getting better at the windows it trains on, and it could be doing that by memorizing them. So after every epoch the training script also measures the loss on the **986** held-out windows, text the model never trains on, and **saves the model only when that validation loss is the best so far**.

It holds back the end of the corpus rather than random windows, on purpose. Neighboring windows share a scene, a speech, sometimes a repeated line, so random held-out windows would mostly be text the model had very nearly seen. The end of the file is most of *The Two Gentlemen of Verona* (from its second scene on), all of *The Winter's Tale* and the poem *A Lover's Complaint*.

The script also lowers the learning rate after every epoch, along a cosine curve: Adam starts at `0.003` in the first epoch and is down to about a tenth of that in the fifth, so the early steps are big and the late ones only polish.

Training on Shakespeare

For this chapter we trained irishQwen itself (the class code, with its exercises filled in) on Shakespeare, with the class tokenizer, the same held-out end of the corpus and the same Adam learning rate and cosine schedule as the training script (lowered a little every step rather than once an epoch). Unfortunately our version of PyTorch is slow, so the model is smaller than the script's: vectors of `128`, `4` blocks, windows of `64` tokens and batches of `16`, `918,912` parameters in all, trained for `3,000` steps (about `1.3` passes through the training text).



The validation loss (measured on `320` held-out windows every `250` steps) starts at `7.20`, worse than the `6.24` of a model that thinks every token is equally likely, since random weights are confidently wrong about everything. It falls below `4` within `250` steps and then keeps creeping down, to `2.59` at step `3,000`.

What a Loss of 2.59 Means

Notice that **the two curves lie almost on top of each other**. The model has seen each training window about once, so it hasn't had the chance to memorize anything, and held-out Shakespeare is just as hard as the Shakespeare it trained on.

A cross-entropy of `2.59` is easier to picture backwards. A model that spread its bets evenly over k tokens would have a loss of $\ln k$, so a loss of `2.59` is as unsure as choosing evenly between $e^{2.59} \approx$ `13` tokens. Our model started out as unsure as picking from all `512` (and then some), and `3,000` steps narrowed that down to about `13`.

Another way to look at it is how often its favorite token is the right one. Over its last `50` training steps the model's top guess was the actual next token `37%` of the time, against `5%` over its first `50`.

That is still a long way from writing Shakespeare. To see how far, we have to make it write.

Generating Text

To write, the model runs in a loop. Give it a prompt, take the probabilities at the last position, pick a token, append it, and run the model again on the longer text. Each new token costs one more run of the whole model. In the browser, starting from the sonnet's first two words:

```python
def sample(probs, temperature=0.8, top_k=10):
    probs = probs ** (1 / temperature)
    p, i = probs.topk(top_k)
    pick = torch.multinomial(p / p.sum(), 1)
    return i[pick].item()

torch.manual_seed(2)
out = list(b"From fairest")
with torch.no_grad():
    for _ in range(40):
        logits = model(torch.tensor([out[-T:]]))
        probs = F.softmax(logits[0, -1], dim=-1)
        out.append(sample(probs))
print(bytes(out).decode(errors="replace"))
```

OUTPUT

```
From fairesta thisth ithorie,
    f  our therus mald
```

The loop only ever shows the model the last `32` bytes (`out[-T:]`), since that is the longest window it trained on. After `100` steps on `20,000` bytes of sonnets the browser model has learned that words are short and separated by spaces, with a comma now and then. That's about all.

Temperature and Top-k

Notice that `sample` doesn't just take the most likely byte. Always taking the top token (greedy decoding) gives the same text for the same prompt every time, and a small model that always takes its favorite tends to fall into a rut. So we draw a token at random, in proportion to its probability, after two adjustments.

Temperature raises every probability to the power `1 / temperature` and normalizes them again. Below `1` the favorites get stronger: three tokens with probabilities `0.5`, `0.3` and `0.2` become `0.54`, `0.29` and `0.17` at a temperature of `0.8`. Above `1` they even out (`0.44`, `0.32`, `0.24` at `1.5`), and at `0` it is greedy decoding.

Top-k throws away everything but the k most likely tokens before drawing, so a rare, silly token can never be picked by bad luck. irishQwen's `sample` does both (plus top-p, which keeps the smallest set of tokens whose probabilities add up to p). The training script samples with a temperature of `0.8` and top-k `40`, and Qwen3-0.6B's recommended settings for chatting are a temperature of `0.7`, top-k `20` and top-p `0.8`.

What Our irishQwen Writes

irishQwen's `chat` method is the same loop, with temperature and top-k, plus one trick to save time: it keeps every block's keys and values from earlier tokens (the KV cache) so each new token only has to run through the model on its own.

Here is our trained irishQwen, the weights from its best validation loss, given the training script's four prompts and allowed `60` tokens each (temperature `0.8`, top-k `40`):

OUTPUT

```
ROMEO. I know I have heard thee out of France, but the honour of hi
BOYET. And I tell you, by thine eye in my rain

So would not do with
```

OUTPUT

```
To be, or not to bear it;
In this excused gams of men throws-
The never true intention
Our brother begins.
POMPEY. We d
```

Plays in the corpus are laid out as a speaker's name in capitals and a period, then their lines, and the model has picked that up, handing over from Romeo to Boyet (a lord from *Love's Labour's Lost*) and from Hamlet's words to a Pompey. The words are nearly all real and the grammar holds for a few words at a time ("I know I have heard thee"), but the sense never lasts a line.

What Our irishQwen Writes, Continued

OUTPUT

```
KING HENRY. O heaven for me, I will remember you.
BASSANIO. Thou that, my lord; I'll be assur'd you. Who can make a
begent cock
```

OUTPUT

```
Shall I compare thee, cold,
Let her with her, by him to choose appear
As we shall speak. Did you be, person me,
Go to the whole ever or ey
```

King Henry gets an answer from Bassanio, from *The Merchant of Venice*, and the sonnet prompt carries on in short lines that start with capitals, like verse, though they scan about as well as they make sense. "gams" and "begent" aren't words, a model that writes in pieces of words can make up new ones. **Our model has learned the layout of Shakespeare, a lot of his vocabulary and a little of his grammar**, from about three million tokens with under a million parameters.

From Shakespeare to Qwen3-0.6B

Our run saw `3,072,000` tokens (`3,000` steps of `16` windows of `64`). The training script's five epochs come to about `12` million. Qwen3-0.6B was trained on about `36` trillion tokens, roughly **three million times as much text as the class model sees**, on a model `175` times the size.

Everything else is the same. irishQwen can load Qwen3-0.6B's real weights (`scripts/import_qwen3.py` converts them), and then the same embedding table, the same `forward` through the same kind of block and the same sampling loop write English, Chinese and Python instead of muttering half-words of Shakespeare. The difference is entirely in the numbers inside the matrices.

That raises an awkward question. Our model's validation loss went down, and its Shakespeare looks a bit more like Shakespeare, but a lower loss on held-out Shakespeare is a pretty narrow idea of "good". Neither our model nor Qwen3-0.6B can be judged by eye forever, and **a loss can't tell us whether a model knows anything**. For that we need a way to measure models against each other.